Universidad Autónoma de Baja California
FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA

Organización y Arquitectura de las Computadoras.

Práctica No. 4:

Captura de texto y palindromo.

Alumno:
Joshua Osorio O. – 1293271
Docente:
José Isabel García R.
07/03/2026

Tabla de contenido

Objetivo 2
Desarrollo 2
Conclusiones 6
Dificultades 7
Bibliografía 8

Objetivo

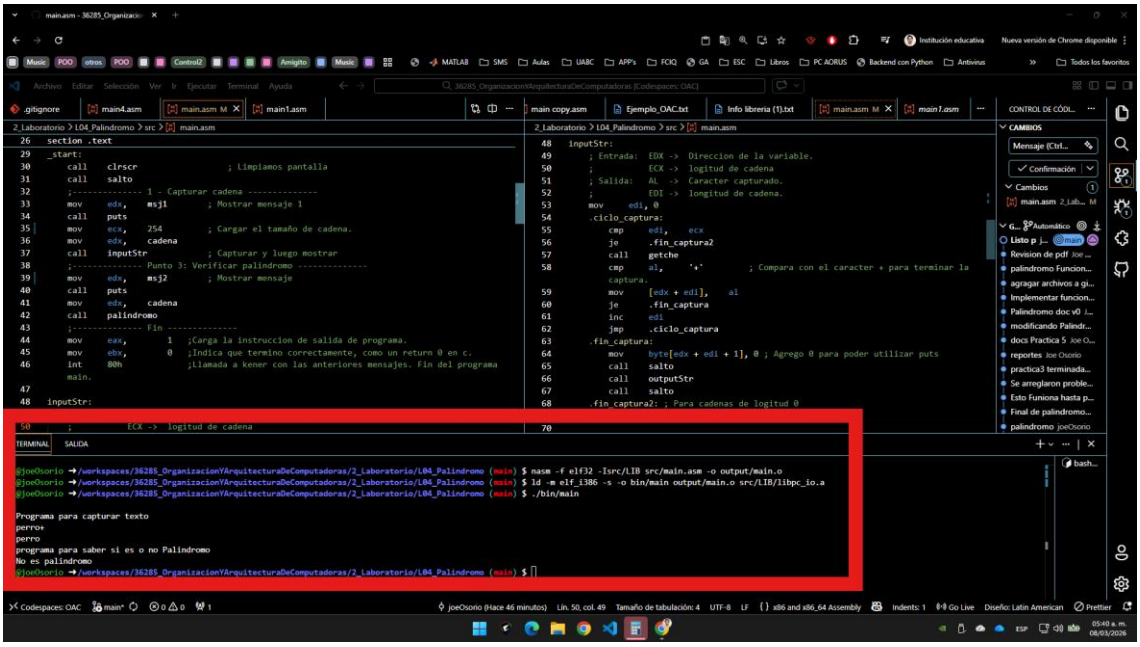
Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Desarrollo

1. (10%) Crear una subrutina que captura caracteres, los almacena en memoria y después mostrar la cadena con puts, recibe en edx la dirección de la cadena a utilizar, debe retornar la longitud de la cadena.

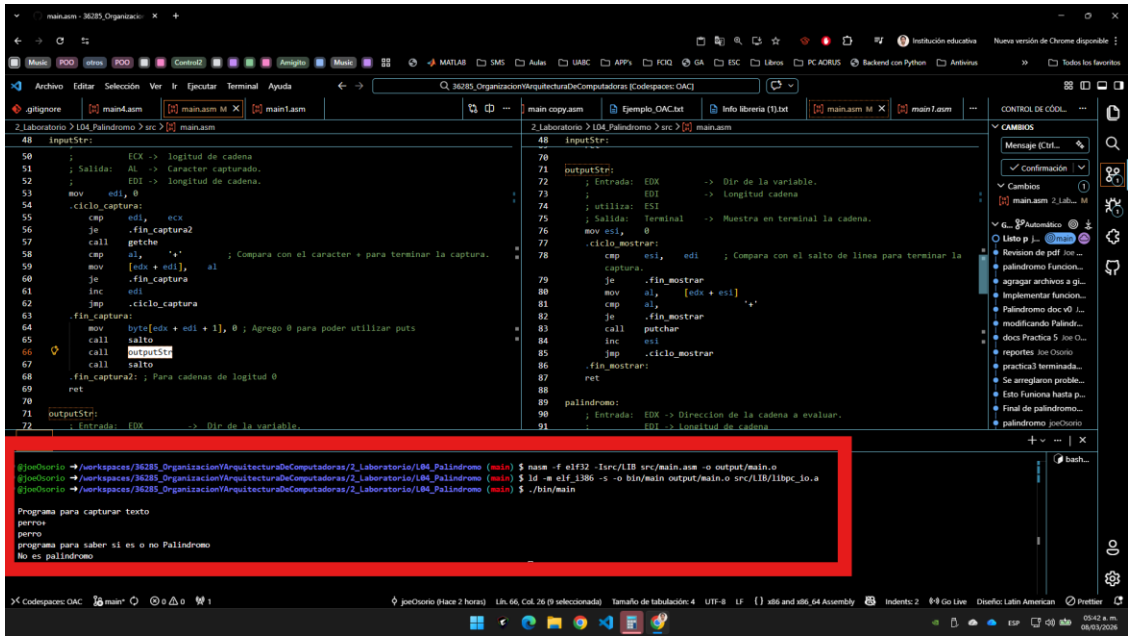
```
;------------- 1 - Capturar cadena -------------
mov     edx,    msj1        ; Mostrar mensaje 1
call    puts
mov     ecx,    254
mov     edx,    cadena
call    inputStr            ; Capturar y luego mostrar
```

```
inputStr:
    ; Entrada:  EDX ->  Direccion de la variable.
    ;           ECX ->  logitud de cadena
    ; Salida:   AL  ->  Caracter capturado.
    ;           EDI ->  longitud de cadena.
    mov     edi, 0
    .ciclo_captura:
        cmp     edi,    ecx
        je      .fin_captura2
        call    getche
        cmp     al,     '+'         ; Compara con el caracter + para terminar la captura.
        mov     [edx + edi],    al
        je      .fin_captura
        inc     edi
        jmp     .ciclo_captura
    .fin_captura:
        mov     byte[edx + edi + 1], 0 ; Agrego 0 para poder utilizar puts
        call    salto
        call    outputStr
        call    salto
    .fin_captura2: ; Para cadenas de logitud 0
    ret
```



2. (20%) Crear una subrutina que muestra cadenas de caracteres en terminal, pueden utilizar cualquier símbolo especial para finalizar la cadena, recibe en edx la dirección de la cadena a imprimir, eso se debe modificar también en la subrutina anterior, finaliza la impresión utilizando el símbolo especial.

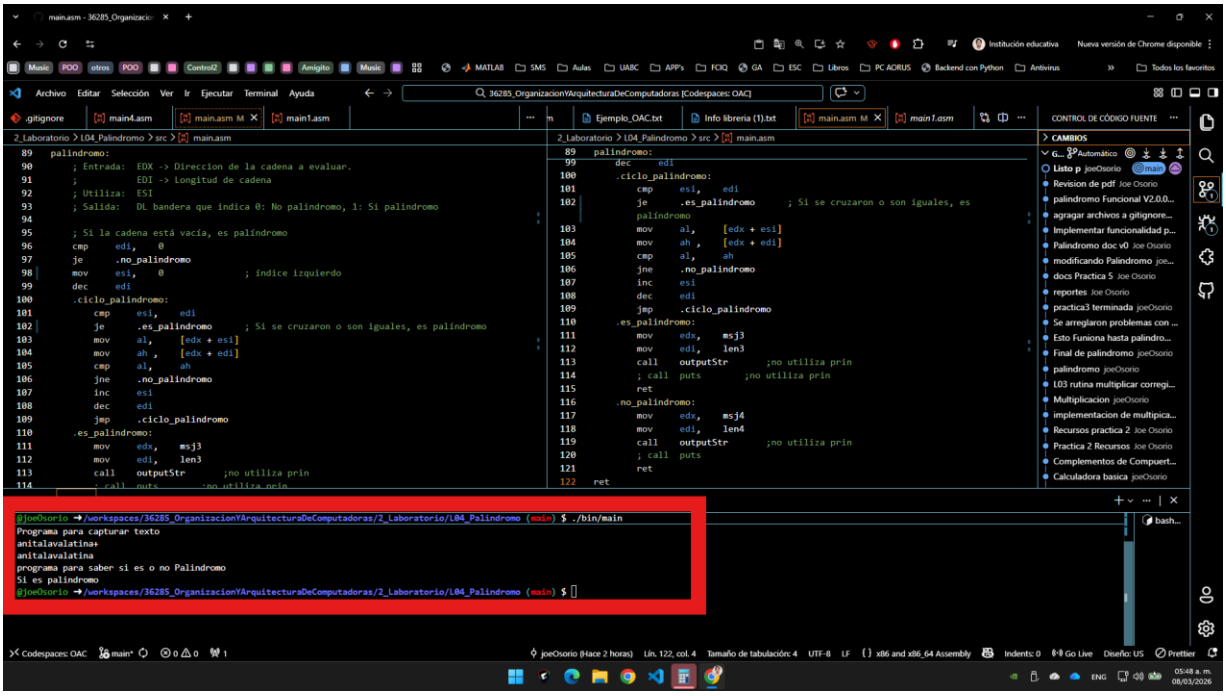
```
outputStr:
    ; Entrada:  EDX          ->  Dir de la variable.
    ;           EDI          ->  Longitud cadena
    ; utiliza:  ESI
    ; Salida:   Terminal     ->  Muestra en terminal la cadena.
    mov esi,    0
    .ciclo_mostrar:
        cmp     esi,    edi     ; Compara con el salto de linea para terminar la captura.
        je      .fin_mostrar
        mov     al,     [edx + esi]
        cmp     al,                 '+'
        je      .fin_mostrar
        call    putchar
        inc     esi
        jmp     .ciclo_mostrar
    .fin_mostrar:
    ret
```



3. (70%) Crear una subrutina que utilizando las instrucciones vistas en prácticas revise si una cadena de caracteres capturada en memoria es un palíndromo o no, debe imprimir un mensaje al final indicando el resultado.

```
palindromo:
    ; Entrada:  EDX -> Direccion de la cadena a evaluar.
    ;           EDI -> Longitud de cadena
    ; Utiliza:  ESI
    ; Salida:   DL bandera que indica 0: No palindromo, 1: Si palindromo

    ; Si la cadena está vacía, es palíndromo
    cmp     edi,    0
    je      .no_palindromo
    mov     esi,    0                   ; índice izquierdo
    dec     edi
    .ciclo_palindromo:
        cmp     esi,    edi
        je      .es_palindromo          ; Si se cruzaron o son iguales, es palíndromo
        mov     al,     [edx + esi]
        mov     ah ,    [edx + edi]
        cmp     al,     ah
        jne     .no_palindromo
        inc     esi
        dec     edi
        jmp     .ciclo_palindromo
    .es_palindromo:
        mov     edx,    msj3
        mov     edi,    len3
        call    outputStr       ;no utiliza prin
        ; call  puts        ;no utiliza prin
        ret
    .no_palindromo:
        mov     edx,    msj4
        mov     edi,    len4
        call    outputStr       ;no utiliza prin
        ; call  puts
        ret
ret
```



Instrucciones vistas en clase y permitidas para esta práctica.

1. add
2. sub
3. mov
4. jmp
5. cmp
6. je
7. loop
8. getch
9. getche
10. puts
11. putchar
12. inc
13. dec

Obligatoriamente de cada paso debe tener las capturas del código y de la terminal.

Conclusiones

En esta práctica se logró implementar exitosamente las tres subrutinas solicitadas: captura de una cadena (inputStr), visualización de esta (outputStr) y detección de palíndromos (palindromo), utilizando únicamente las instrucciones permitidas (add, sub, mov, jmp, cmp, je, loop, getch, getche, puts, putchar, inc, dec).

Se puso en práctica el direccionamiento indirecto con registros base más desplazamiento ([edx + edi]), lo que permitió acceder de manera eficiente a cada carácter de la cadena. Además, se comprendió la importancia de manejar correctamente los registros de propósito general (eax, ebx, ecx, edx, esi, edi) para no interferir con datos necesarios en otras partes del programa como al momento de utilizar subregistros como al, ah, etc.

La subrutina inputStr demostró cómo capturar caracteres hasta un símbolo especial (+) y almacenarlos en memoria, añadiendo al final un terminador nulo para compatibilidad con futuras impresiones. Por su parte, outputStr mostró cómo recorrer e imprimir una cadena carácter por carácter hasta encontrar el delimitador o alcanzar la longitud indicada.

Finalmente, palindromo evidenció la utilidad de los índices izquierdo (esi) y derecho (edi) para comparar desde los extremos y determinar si una cadena es o no un palíndromo, imprimiendo el mensaje correspondiente mediante nuestra propia rutina de visualización, cumpliendo así con el objetivo de no depender de funciones externas como puts.

Con esta práctica se afianzaron conceptos fundamentales de ensamblador: manejo de memoria, modos de direccionamiento, control de flujo con saltos condicionales y comunicación entre subrutinas a través de registros.

Dificultades

Manejo de índices y comparaciones

Al principio resultó complejo utilizar los registros edi y esi porque nunca se preguntó dónde se inicializaban y se daba por hecho que al momento de utilizar getche se iniciaba además de coordinar los registros esi y edi para recorrer la cadena desde ambos extremos sin perderse, especialmente al momento de detener el ciclo cuando los índices se cruzaban.

Se solucionó utilizando cmp esi, edi y saltos condicionales (je, jne) para decidir cuándo terminar.

Uso del delimitador personalizado

La decisión de usar el carácter '+' como fin de captura, en lugar del tradicional Enter, obligó a modificar tanto inputStr como outputStr para que ambas reconocieran el mismo símbolo.

Fue necesario asegurar que el terminador nulo (0) se agregara después del último carácter capturado, para poder utilizar puts de la librería sin problemas.

Coordinación entre subrutinas

Al principio, inputStr no retornaba la longitud de la cadena, lo que dificultaba su uso en palindromo. Se corrigió almacenando la longitud en edi y manteniendo ese valor para la siguiente rutina.

También hubo que tener cuidado de no modificar registros que contenían valores importantes (como la dirección de la cadena) sin antes preservarlos.

Control de saltos y bucles

En ocasiones los bucles no terminaban correctamente debido a comparaciones mal planteadas o a la omisión de incrementos/decrementos.

Se depuró cada ciclo verificando manualmente los valores de los registros con pequeños mensajes de prueba (aunque no se incluyen en el código final).

Compatibilidad con la librería pc_io.inc

Aunque se usaron las funciones getche, putchar y puts de la librería, hubo que entender cómo afectaban los registros (por ejemplo, puts modifica edx) para no perder datos importantes.

Se optó por guardar en push/pop los registros necesarios en las subrutinas críticas.

A pesar de estas dificultades, se logró un programa funcional que cumple con todos los requisitos de la práctica, consolidando así los conocimientos de programación en ensamblador y manejo de

Bibliografía

[1] D. A. Alpern, “El microprocesador 80386.” https://www.alpertron.com.ar/80386.HTM
[2] “Guide to x86 Assembly.” https://www.cs.virginia.edu/~evans/cs216/guides/x86.html